Validation day guide — how every part works, where it appears, what to show

For each part: **what to say** in one sentence, **how it works**, **where it appears** (page of the app, file, tool), **what to show** the tutor click by click, the **numbers** to quote, the **questions** the tutor may ask with short answers, and the **code** to open if asked.

---

0. The map

0.1 The whole flow in one picture

The numbers are the parts of this guide.

One workflow per application, from the CV to the final decision (n8n workflow *Application pipeline*, one node per step; part 3 of this guide):

```
CV (PDF, scan, photo, Word; EN/FR) — candidate portal or recruiter upload
 8  N8N "Application pipeline" ─► intake checks (consent, type, size)
 1  DOCUMENT PROCESSING: PyMuPDF text layer | OCR (PaddleOCR) | python-docx  →  text
 1  EXTRACTION: Qwen2.5-7B + our QLoRA adapter (part 2) on the Kaggle GPU → JSON → checks → profile
 7  STORE: PostgreSQL (candidates, applications…)   6  INDEX: Qdrant (BGE-M3 vectors) → the candidate is answered
 3  SCREENING AGENT (LangGraph): understand the task → plan the searches → act → evaluate & rerank → iterate
 4     its RAG: CV passages + HR rules (BGE-M3 + Qdrant) → reranking (BGE cross-encoder) → contextualization
 5     its analysis: Qwen2.5-7B judges every requirement with a CV quote → code checks every claim → rubric grade
    OUTPUT: evidence-backed report (screen, PDF) → recruiters notified
    HUMAN VALIDATION: the workflow WAITS until a recruiter decides → final decision → answer to the candidate → closed
```

Also: job advertisements read by Qwen (part 1); the Matching page (fast ranking and the fixed analysis: parts 4, 5); the AI assistant (a second LangGraph agent with 8 tools: part 3); n8n messages for the other events (part 8).

0.2 The app's pages

Menu	Page	Parts of this guide
Overview	Dashboard · **Application pipeline**	setup, counts; every application through its 12 steps, the screening agent's path (3)
Recruiting	Candidates · Jobs	1 (CVs go into the pipeline; job ads read by Qwen)
Recruiting	Matching · Reports · Human validation	4, 5 (a decision resumes the application's workflow: 3, 8)
AI	AI assistant · Search	3, 4, 6
Automation	Candidate portal · Notifications & n8n	8
Under the hood	Architecture (live) · Document processing · Vectors & RAG · Matching internals · Assistant internals · Database	every part, from the inside
Results	Evaluation	1, 2 (and 3, 4, 5 once their Kaggle jobs have run)

Also: API documentation http://localhost:8000/docs · Qdrant dashboard http://localhost:6333/dashboard · n8n http://localhost:5678 · Adminer http://localhost:8080 · Kaggle notebooks (fine-tuning, AI server).

0.3 Preparation

The days before

- Run the evaluations that are not done yet, so the Evaluation page has every number (each needs your go, it uses Kaggle quota): `python scripts/kaggle_run.py all rag` (~15 min), `all matching` (~1 h), `all screening` (~1 h, the screening agent next to the fixed analysis), `all agent` (~30 min); optional `all extraction-vlm` (~2–3 h, the vision model, to show multimodality with numbers).
- Do the whole test plan once (`docs/reports/test_plan.md`).

- Take screenshots of the key screens (a backup if the Kaggle session fails on the day).

30 minutes before

1. Start the Kaggle AI server notebook (*Run All*, ~5 min), paste the 4 lines into `.env`.
2. `docker compose up -d postgres qdrant n8n` and `docker compose --profile tools up -d adminer`.
3. Start the API (`--host 0.0.0.0`) and Streamlit; check the sidebar: 4 green dots.
4. Pre-run what is slow, so the demo stays fast: one application through the portal (via n8n) that waits for a decision, one already decided (closed), 3 analyses for *GenAI Engineer* (Matching), 2–3 assistant questions, one *Document processing* comparison on your CV (that page keeps its result only while the browser tab stays open).
5. Open these tabs: the app (Architecture page), n8n (Workflows), Qdrant dashboard, Adminer, Kaggle (the fine-tuning notebook), the Evaluation page.

0.4 A 20-minute demo script

Time	Show	Part
0–2	Under the hood → **Architecture (live)**: the diagram, every part green; the flow in one sentence: one workflow per application	all
2–6	**Candidate portal** → apply through n8n → n8n *Application pipeline* canvas and its execution waiting at the Wait node → **Application pipeline** page: the steps, the screening agent's path and tool calls	3, 8, 1
6–8	Under the hood → **Document processing**: the scanned PDF → OCR pages → prompt → Qwen's raw JSON; base vs fine-tuned on your CV; **Evaluation** → before/after fine-tuning	1, 2
8–11	**Vectors & RAG** → *One search, step by step*; **Reports** → the application's analysis: passages per requirement, the agent's checks, the PDF	4, 5, 6
11–14	**Human validation** → *Invite to an interview* → n8n: the same execution resumes and closes → **Notifications**: the invitation	9, 8, 7
14–16	**Matching** (ranking in seconds, the fixed analysis) and **Evaluation** → matching, screening agent	4, 5
16–18	**AI assistant** / **Assistant internals** → a question → its path; one refusal of a discriminatory question	3
18–20	**Database** (`applications` step log, reports, traces) and Adminer; conclusion: limits and next steps	7

---

1. Extraction and document processing (multimodality)

Say it: *"Any CV — digital PDF, scan, photo or Word, in English or French — becomes a validated structured profile: we read the document on our side (text layer, or OCR for scans), our fine-tuned Qwen2.5-7B fills a JSON schema, and code checks and normalises the answer."*

How it works

1. **Upload** (Candidates page, candidate portal through n8n, or the API): type checked (pdf, docx, png, jpg, jpeg), size ≤ 15 MB, safe file name. The file is processed in a temporary folder and deleted at once: only the profile and the CV text are kept (candidate data policy KB-HR-004).
2. **Reading** (`ingestion/parser.py`):
   - PDF: PyMuPDF reads each page's text layer. A page with fewer than 30 characters is a scan: it is rendered as an image and read by **OCR** (RapidOCR running PaddleOCR PP-OCRv5 models on ONNX Runtime, on our PC);
   - Word: python-docx (paragraphs and tables); images (PNG, JPG): OCR;
   - the text is normalised (NFKC: "fi" ligatures, full-width characters; French accents kept);
   - the input type is recorded: `text_pdf`, `scanned_pdf`, `mixed_pdf`, `docx`, `image`.
3. **Prompt** (`extraction/extractor.py`): a system prompt with **8 rules** — (1) only what the CV says, never invent; (2) null when absent; (3) the candidate's own contact details, city only; (4) every position, and the employer is not a project name; (5) one entry per degree; (6) skills from anywhere in the CV, but group headings are not skills; (7) projects with their technologies; (8) certifications, and spoken languages only — plus a worked example (an invented CV, "Jane Doe"). Then a user message with the CV text (≤ 16,000 characters) and the **JSON schema** of the answer (`CVExtraction`). The fine-tuning used exactly this prompt.
4. **Generation**: Qwen2.5-7B-Instruct **with our QLoRA adapter**, on the Kaggle AI server (2 × T4, float16, greedy decoding = deterministic, ≤ 2,048 answer tokens). A generation is a job polled every 3 s, because the Cloudflare link closes requests longer than ~100 s.
5. **Robust JSON** (`llm/huggingface_adapter.py`): the JSON object is taken out of the answer; one cut by the token limit is closed; it is validated by a lenient Pydantic schema (`CVExtraction`: numbers accepted as text, null lists accepted, lists capped at 15 positions, 8 degrees, 60 skills…).
6. **Normalisation** (`to_profile`): dates normalised (`2021-01`, `2021` or `present`), empty values and duplicates removed, a "skill" written as a whole sentence split into the known skill names it contains (looked up in an **English/French skill lexicon**: ESCO v1.2.1 + our own list), project technologies added to the skills, group headings copied as skills removed → `CandidateProfile`.
7. **Long CVs**: if the GPU runs out of memory, the CV is retried once, cut to 8,000 characters.
8. **Stored and indexed**: a row in PostgreSQL (part 7) and its snippets in Qdrant (part 6).

Multimodality, two ways:

- *in the app*: several input modalities (digital text, scanned pages, photos, Word) through OCR → the text model;
- *the vision-language model*: Qwen2.5-VL-7B-Instruct reads the page images directly, without OCR (up to about 1 million pixels, i.e. 1,280 image tokens, per page): implemented (`CVExtractor.extract_images`); its evaluation is the Kaggle job `extraction-vlm`, **not run yet**. A third engine, `extraction-text` (Qwen on the dataset's clean text), would measure what OCR costs (not run yet either).

Where it appears

Where	What you see
Under the hood → Document processing	step by step: each page read by text layer or OCR (with the OCR confidence), the extracted text, the system prompt and the user message, Qwen's raw JSON, the final profile
Candidates → *Add a candidate*	the profile, and a caption: input type, model, *fine-tuned adapter*, timings
Candidates → *Test the extraction only*	the same, nothing stored
Candidate portal	a candidate applies; n8n sends the CV to the API

Where	What you see
Under the hood → Database → `candidates`	`profile` (JSON), `cv_text`, `extraction` (model, adapter, seconds, input type)
Evaluation → *DSO1 · Extraction*	the scores on the frozen evaluation set

Show the tutor (3 minutes)

1. Under the hood → **Document processing** → `build/test_files/syn05_scanned.pdf` (a scan with no text) → *Process this CV step by step*. Point at: the page read by **OCR (scan)** and its confidence; the extracted text; the 8 rules and the JSON schema; Qwen's raw JSON; the clean profile.
2. Candidates → *Add a candidate* → `data/processed/eval_v1/synthetic/files/syn10.docx`: a French Word CV, the values stay in French.
3. Under the hood → **Database** → `candidates` → open the row: the profile JSON and the extraction details.

Numbers to quote (frozen dev set, 71 documents, with the fine-tuned adapter)

Overall F1	Experience F1	Dates exact	Education F1	Skills F1	Identity accuracy	Invented identity fields	Time
81.2 %	92.9 %	97.6 %	88.8 %	62.0 %	94.6 %	5 (in the 44 CVs with labelled contact details)	72 s per CV

By document type: real scanned PDFs 80.2 % (60 documents, the hard case: OCR on old scans), digital PDFs 94.5 % (6), Word 93.8 % (5); French CVs 90.9 % (5). One document failed in each run (not the same one): it is scored as an empty profile, so it lowers the score.

Questions the tutor may ask

- *Why OCR + a text model rather than the vision model?* — Text is cheaper for the model than page images (up to 1,280 image tokens per page), the OCR runs on our PC, and every step can be checked (the text Qwen read is shown); the vision path is implemented, and its evaluation job (`extraction-vlm`) is ready to compare the two.
- *How do you stop invented values?* — Rules 1–2 of the prompt, deterministic decoding, a strict schema, and the evaluation counts identity values not found in the CV ("hallucinations"): 12 → 5 with fine-tuning.
- *What if the model writes broken JSON?* — It is repaired when possible (cut answers are closed) and validated; otherwise the error is reported, nothing crashes.
- *Why is the CV file deleted?* — It is personal data (KB-HR-004): we keep only what the screening needs.

Code: `src/recruitment_ai/ingestion/parser.py`, `ingestion/ocr.py`, `extraction/extractor.py`, `extraction/schemas.py`, `llm/huggingface_adapter.py`, `llm/remote_adapter.py`, `llm/server.py`.

---

2. Fine-tuning vs baseline

Say it: *"We taught Qwen2.5-7B our exact extraction task with QLoRA, on 873 CVs: we train only 0.53 % of the weights — a 173 MB adapter — on Kaggle's free GPUs in 2 h 22 min. On the frozen dev set, the overall F1 goes from 78.9 % to 81.2 % and invented identity fields drop from 12 to 5."*

How we did it

1. **Data** (`training/data.py`): the public Resume Parser dataset (Hugging Face), *train* split: 12,440 CVs with reference profiles. Removed: CVs also in our evaluation splits (306: no leakage), labels poorly grounded in their CV (< 80 % of values found: 391), repeated texts (541), corrupted (36), too long (98) → 11,068 candidates → **1,000 training CVs** + 20 held out.
2. **Format**: each example = the system prompt + the user message, built by **the same function as at inference** (`structured_prompt`), + the reference JSON as the answer. The loss is computed on the answer tokens only.
3. **QLoRA** (`training/qlora.py`):
   - the base model is loaded **frozen, in 4-bit NF4** with double quantization (bitsandbytes); computations in float16 (the T4 has no bfloat16);
   - small trainable **LoRA** matrices are added to the 7 projections of every layer (q, k, v, o, gate, up, down): rank 16, alpha 32, dropout 0.05 → **40.4 M trainable parameters = 0.53 %** of the 7.6 B;
   - gradient checkpointing; optimizer paged AdamW 8-bit; learning rate 2e-4, cosine schedule, 3 % warmup; gradient clipping 0.3; **1 epoch**; sequences ≤ 4,096 tokens (127 longer examples skipped → **873 used**);
   - batch 1 × 4 accumulation × **2 GPUs** (DDP with torchrun) = 8 → **110 steps** in **141.5 minutes**.
4. **Built for Kaggle's limits** (`notebooks/kaggle/finetune_qwen_qlora.ipynb`, 9 cells: parameters, checks, code, earlier progress, data, model weights, training, quick check, export): a checkpoint every 15 minutes; training saves and stops by itself 40 minutes before the session limit; a new run finds the earlier progress (a finished adapter or the last checkpoint, in the session or in the previous run's output) and resumes with the same examples in the same order. Problems we solved:
   - training lines cut in half by rare Unicode separators (U+2028) inside some CVs → split on real line breaks only;
   - out of GPU memory → logits computed for the answer tokens only (4,096 positions × Qwen's 152k-token vocabulary is several GB), and no layer cast to float32;
   - an old library in Kaggle's image (torchao) made PEFT refuse to load the adapter on a float16 model → uninstalled.
5. **Result**: loss 0.40 → 0.18; the adapter (173 MB) is the output of the Kaggle notebook.
6. **Evaluation**: the same frozen dev set, the same code and scoring, only the adapter differs (job `extraction-ft`).
7. **Use**: the AI server loads the adapter. CV extraction uses it; every other task (job reading, analysis, agent) switches it off for that call (`PeftModel.disable_adapter`), because it was trained for one task only.

Results (dev split, 71 documents)

Metric	Base Qwen2.5-7B	Fine-tuned	Change
Overall F1	78.9 %	**81.2 %**	+2.4 pts
Experience F1	88.2 %	**92.9 %**	+4.7
Dates exact	95.7 %	**97.6 %**	+1.9
Education F1	**92.3 %**	88.8 %	−3.5
Skills precision / recall / F1	67.8 / 47.7 / 56.0 %	**72.5 / 54.2 / 62.0 %**	+6.0 F1
Identity accuracy	93.2 %	**94.6 %**	+1.4
Invented identity fields (44 CVs with labelled contact details)	12	**5**	−7
Seconds per CV	90.4	**72.1**	−18 s (shorter answers)

Per group: real scanned CVs 77.2 → **80.2 %** (60 documents); synthetic CVs 97.3 → 94.1 % and French CVs 94.8 → 90.9 % (11 and 5 documents, already near the ceiling). Per CV (change above 0.5 point): **40 better, 22 worse, 9 unchanged**. The test split has not been used yet: it is kept for the final evaluation.

Where it appears

Where	What you see
Evaluation → DSO1 · Extraction (before/after fine-tuning)	metric cards with the change, a bar chart (0–100 %), results by document type, CV by CV
Evaluation → DSO1 · Fine-tuning run	the loss curve and the training report
Under the hood → Document processing (box *also run the base model* ticked)	the same CV through both models, live: sections found, identity, skills only one model found, both raw answers and timings
Under the hood → Architecture (live) · Dashboard	the box *4. Fine-tuned Qwen2.5-7B* shows *QLoRA adapter loaded*; the Dashboard's status says + *fine-tuned adapter*
Files	`data/evaluation/finetune/` (report, data selection, parameters, loss), `data/evaluation/extraction/` (both runs)
Kaggle	the notebook *recruitment-ai-finetune* and its output (the adapter)

Show the tutor (4 minutes)

1. **Evaluation** → *before/after*: the cards (+2.4 pts overall, 12 → 5 invented fields), the chart, *Per CV*.
2. *Fine-tuning run*: the loss curve.
3. Under the hood → **Document processing** → your CV, comparison ticked: the table base vs fine-tuned.
4. If asked "how": open the notebook on Kaggle (cells 1, 5, 7) or `src/recruitment_ai/training/qlora.py`.

Questions the tutor may ask

- *Why QLoRA and not full fine-tuning?* — Full fine-tuning of 7 B parameters needs over 100 GB of GPU memory. QLoRA freezes the base model in 4 bits and trains 0.53 % of the weights: it fits two 16 GB T4s, takes 2.4 h, gives a 173 MB adapter, and the base model stays usable (adapter on/off per request).
- *What are rank and alpha?* — Each adapted weight W becomes W + (alpha/r)·B·A, with A and B thin matrices of rank r = 16; alpha/r = 2 scales the update.
- *Why NF4?* — A 4-bit format designed for normally distributed weights; double quantization also compresses the quantization constants.
- *Why only 1 epoch and 1,000 CVs?* — The GPU quota (30 h per week) and silver labels; the loss dropped fast (0.40 → 0.18 in the first 20 steps), then stayed between 0.13 and 0.21.
- *Why did education drop?* — The adapter learned the education style of the dataset's own labels; the samples are small; next step: more education-rich examples or cleaner labels.
- *How do you know it isn't tuned on the test?* — Evaluation CVs (306) were removed from the training data; the evaluation set is frozen (one checksum per file, verified before each run); all the numbers above are on dev, and the test split is untouched, kept for the final evaluation.

Code: `src/recruitment_ai/training/data.py`, `training/qlora.py`, `scripts/finetune_qlora.py`, `notebooks/kaggle/finetune_qwen_qlora.ipynb`, `src/recruitment_ai/evaluation/compare.py`.

3. Agents and orchestration

Say it: *"Every application goes through one workflow, from the CV to the final decision. n8n runs it, one node per step, and its agent node is a LangGraph screening agent: given the job, it understands the task, decides which searches to run, retrieves and reranks the evidence, searches again while must-haves lack evidence, then Qwen judges every requirement and code checks every claim. The workflow then waits for the recruiter's decision and answers the candidate. A second LangGraph agent, the assistant, answers recruiters' questions."*

3.1 The workflow: one per application (`services/pipeline.py`)

#	Step	Part of the diagram	What runs
1	Intake checks	1-2	consent (KB-HR-004), file type and size; the application is registered
2	Document processing	3	PyMuPDF text layer · python-docx · OCR of scans (PaddleOCR); the file is then deleted
3	Candidate JSON	3-4	Qwen2.5-7B + our QLoRA adapter → structured profile
4	Store and index	8, 5	PostgreSQL (`candidates`) + Qdrant (BGE-M3 snippets)
5	Acknowledge the candidate	7	the candidate's answer ("within 10 working days")
6	Screening agent	5	the LangGraph agent below → one report per job
7	Evidence-backed report	6	the analysis stored, Markdown and PDF
8	Notify the recruiters	7	review needed (priority for grades 2-3)
9	Human validation	9	**the workflow waits** until a recruiter decides
10	Final decision	9	interview, shortlist or rejection (on hold keeps waiting)
11	Notify the candidate	7	interview invitation or answer (e-mail node ready)
12	Close the application	8	final status and timings

- **Same steps, two runners**: n8n (workflow *Recruitment — Application pipeline*) calls `POST /applications/{id}/steps/{step}` node by node; without n8n, the API runs the steps itself in the background. Each step is logged on the application (status, time, result, error).
- **The wait**: an n8n *Wait* node pauses the execution; its resume URL is given to the API at step 8; the recruiter's decision (Human validation) calls it, and the same execution goes on. If n8n does not answer, the API finishes the steps itself. After 10 days without a decision, n8n reminds the recruiters and waits again.
- **Robust**: a step out of turn is refused (409); a step already done answers its stored result (a retried node never runs twice); a failed step is recorded and notified, and the workflow continues from it once fixed.
- **Fine-tuning is not a step**: it ran once on Kaggle; step 3 uses the adapter it produced.

3.2 How the screening agent works (`agents/screening.py`)

1. **Its task comes from the workflow**: the job the candidate applied to; for a spontaneous application, the open jobs graded 2 or more by the fast matching (at most 2), else the best one, so that a recruiter can answer.
2. **The graph** (LangGraph `StateGraph`), the seven steps of the architecture's agent box:

```
START → understand → plan → act → evaluate ──(weak must-haves, rounds left)──→ plan
                                          └──(evidence enough)──→ analyse → verify → next_job → (plan | END)
```

Node	Step	What happens
`understand`	1. Understand the task	the job(s) to analyse (semantic matching tool for a spontaneous application)

Node	Step	What happens
`plan`	2. Decide which tools to use	round 0, by code: the CV searched for every requirement + the job's HR rules. Later rounds, by Qwen: for each weak requirement, which search — the CV again with other words, or the HR rules (accepted equivalences)
`act`	3-4. Call the tools, retrieve	CV search tool (the CV text cut into passages tagged with their section, BGE-M3 then the BGE reranker), HR knowledge search tool (Qdrant)
`evaluate`	5-6. Evaluate & rerank, iterate	the best 3 passages per requirement; must-haves still without a strong passage from a position, project or degree → plan again
`analyse`	7. Final analysis	Qwen2.5-7B judges every requirement from its passages and the CV: level, quote, years
`verify`	7. Checks by code	the quote must be in the CV and support the requirement (cross-encoder); a quote found only in a skills section or the summary → "listed"; strong evidence the model left out is used; years counted from the relevant positions; the rubric gives grade, score and action

3. **Guards by code**: the systematic searches always run; at most 4 tool calls per round and 2 planning rounds; a repeated search, an unknown tool or a search on a protected attribute is refused (and recorded); a failed plan falls back to HR-rule searches, a failed analysis to the fast semantic estimate.
4. **Traceability**: every report keeps the agent's trace — its timeline (each node, its time and tokens), every tool call, the passages kept for each requirement, the checks that changed the model's judgement.

3.3 The AI assistant (`agents/agent.py`)

1. **The graph** (LangGraph `StateGraph`): three nodes run in a loop.
   - **decide**: Qwen2.5-7B (base model) receives its system prompt (the 8 tools and the rules), the question, the jobs, and the results of the tools already called. It answers JSON: `{"thought", "action", "arguments"}`.
   - **act**: the chosen tool runs; its result becomes an *observation* added to what Qwen sees next.
   - back to **decide**, until Qwen chooses `final_answer` or 4 tool calls are done;
   - **respond**: Qwen writes the answer from the observations only, naming the candidates, jobs and HR documents.
2. **The 8 tools** (`tools/registry.py`), all read-only:

Tool	What it does
`search_candidates`	semantic search in all CVs (Qdrant + reranker), best evidence per candidate
`search_jobs`	the same in the job descriptions
`search_hr_knowledge`	the HR knowledge base (policies, rubric, job profiles, skills)
`get_candidate` / `get_job`	a profile / a job from PostgreSQL (no contact details)
`rank_candidates`	all candidates of a job, graded with the rubric
`match_candidate`	one candidate vs one job, requirement by requirement
`rerank_candidates`	orders candidates on one precise criterion with the cross-encoder

3. **Guards by code**: arguments validated (common variants accepted: `q` for `query`; values clamped), the same call never repeated, an unknown tool or a failure becomes an observation, at most 4 tool calls, and if the model fails to answer the tool results are shown as they are.
4. **Traceability**: every run is stored (`agent_runs`) with its steps and its **timeline** (each node, its decision, time and tokens).

Where it appears

Where	What you see
Application pipeline	every application, its 12 steps colored by state; each step's result; the screening agent's **path through its graph** (nodes, rounds, times, tokens) and every tool call
Reports → an analysis → *Screening agent*	the passages kept per requirement, the planning rounds, the checks that changed the model's judgement
n8n → *Application pipeline* → **Executions**	one execution per application: green up to the Wait node, then the last steps after the decision
AI assistant	the chat; example questions; *How it answered*: each tool call with its arguments and result
Under the hood → **Assistant internals**	the assistant's graph; the **path** of a question through it; the tools; the two system prompts
Under the hood → Database → `applications`, `agent_runs`	each application's step log; every question with its steps and timeline (JSON)
http://localhost:8000/docs	`POST /applications`, `POST /applications/{id}/steps/{step}`, `POST /agent/ask`…

Show the tutor (4 minutes)

1. **Candidate portal** → a job, a CV, consent, *Through n8n* → the answer; n8n → *Application pipeline* → **Executions**: the run waits at *Wait for the recruiter's decision*.
2. **Application pipeline** → the application: the steps turned green; open *Screening agent*: the graph path (understand → plan → act → evaluate → … → verify), the tool calls, the checks.
3. **Human validation** → *Invite to an interview* (a date) → n8n: the same execution goes on to *Close the application*; **Application pipeline**: every step done; **Notifications**: the invitation prepared.
4. **AI assistant**: *"Which of our candidates are women? List their names."* → refusal citing the fair evaluation policy (KB-HR-003).

Questions the tutor may ask

- *Why is fine-tuning not a step of the workflow?* — Training runs once, offline (110 steps on Kaggle); what it produced, the adapter, is used at step 3 for every CV. Fine-tuning per CV would be neither possible nor useful.
- *Is the screening agent really an agent, or a fixed chain?* — Its first round is fixed on purpose (the minimum evidence is always gathered); then Qwen decides which searches to run for the weak requirements and when to stop, within limits set by code. The fixed chain still exists (Matching page) and is its baseline in the evaluation.
- *Why not let Qwen decide everything?* — A 7B model plans unreliably: the code guarantees the systematic searches, refuses repeated or protected searches, and checks every claim; Qwen decides where its judgement adds value (other words to search, the level of each requirement).
- *What if the model fails in the middle?* — A failed plan → HR-rule searches; a failed analysis → the fast semantic estimate with a warning; a failed step → recorded, notified, continued from that step.
- *What if the recruiter takes days?* — The n8n execution waits (it is stored by n8n); after 10 days it reminds the recruiters (service level KB-RR-001) and waits again.
- *Why LangGraph?* — An explicit state machine: named nodes, conditional edges, a loop limit; easy to trace and test. (The same nodes also run in a plain loop if LangGraph is missing.)
- *How does Qwen call tools without native function calling?* — It writes a JSON plan or decision; the code validates it with Pydantic, runs the tools and feeds the results back.
- *Can an agent take decisions?* — No: their tools only read, their prompts forbid hiring decisions, and every decision is a recruiter's (Human validation).
- *How are they evaluated?* — The screening agent on the 36 graded pairs, next to the fixed analysis (Kaggle job `screening`); the assistant on 20 scripted questions: right first tool, expected answer, candidates named only if a tool returned them, refusals (Kaggle job `agent`).

Code: `src/recruitment_ai/services/pipeline.py`, `agents/screening.py`, `matching/cv_index.py`, `api/routers/applications.py`, `n8n/workflows/1_application_pipeline.json`; the assistant: `agents/agent.py`, `tools/registry.py`, `services/assistant.py`, `api/routers/agent.py`, `evaluation/agent_eval.py`.

---

4. Agentic RAG: embeddings and retrieval, reranking, contextualization

Say it: *"Every text becomes a 1,024-number vector with BGE-M3 and is stored in Qdrant; a question retrieves the closest texts, a cross-encoder reranks them precisely, and the best ones are assembled with the job, the computed experience and the CV into the context Qwen reasons on. It is agentic because the screening agent decides itself which further searches to run and when it knows enough (as does the assistant for its questions)."*

How it works

1. **Indexing** (`rag/knowledge_index.py`, `rag/search_index.py`): three Qdrant collections, cosine distance:
   - `hr_knowledge`: the 15 HR documents cut by section (one rule = one chunk, ≤ 300 words, prefixed with the document title) → **99 chunks**;
   - `candidates`: each CV as snippets (one per position, project, degree, certification, group of 30 skills, the languages, the summary) → 77 for the 12 demo candidates;
   - `jobs`: each job's requirements + its description by paragraphs → 27 for the 5 jobs;
   - each point has a payload (text, document id, section, role, language, candidate or job id) and a stable id, so indexing again replaces instead of duplicating.
2. **Embeddings**: **BGE-M3** (BAAI/bge-m3), 1,024 dimensions, multilingual (a French CV and an English job land in the same space), computed on the Kaggle GPU in float16 (`/embed`, batches of 64).
3. **Retrieval**: the question → its BGE-M3 vector → Qdrant returns the closest points by cosine similarity (20 chunks for the HR knowledge, at least 30 snippets for CVs and jobs, 15 on the step-by-step page), optionally filtered (job profile, language).
4. **Reranking**: **BGE-reranker-v2-m3**, a *cross-encoder*, reads each (question, text) pair together and gives a relevance probability (0–1); the points are reordered and the best kept. For candidate and job search, the best snippet per candidate (or job) is kept, then reranked.
5. **Contextualization** (`matching/analyzer.py`): for an analysis, 3 searches in the HR knowledge base — the job's profile and accepted equivalences (filtered on the job's role), the scoring rubric, the experience rules — the best 2 of each, deduplicated; then one prompt with the requirements (must/nice, importance, minimum years), those company rules, the **experience timeline computed by code** (KB-RR-003) and the CV text (≤ 7,000 characters).
6. **Agentic**: in the pipeline's screening agent, every requirement is first searched in the candidate's CV (the CV text cut into passages tagged with their section, `matching/cv_index.py`: BGE-M3, then the reranker) and the job's rules in the HR knowledge base; then Qwen reads the weak requirements and decides which further searches to run (other words in the CV, accepted equivalences in the HR rules), and the agent searches again while must-haves lack evidence (part 3). In the assistant, Qwen chooses among CV search, job search, HR knowledge search, ranking, matching and reranking. The fast matching also uses retrieval + reranking: for each requirement, the closest CV snippets are reranked to find the evidence.

Where it appears

Where	What you see
Under the hood → Vectors & RAG → *Collections*	the 3 collections: points, dimension 1024, cosine
→ *Browse points*	any point: its payload and its 1,024 numbers (drawn as a line)
→ *2D map of the vectors*	the vectors flattened to 2D (PCA): similar texts form groups (color by document, candidate, section)
→ *One search, step by step*	the query vector, the cosine order, then the reranker order with ↑/↓ moves and the points kept
Under the hood → Matching internals	for each requirement, the CV snippets compared with cosine and reranker scores; the 3 HR searches and their results; the full prompt
Search	HR knowledge, candidates and jobs search with scores
Reports → *Company rules used*	the HR documents (ids) given to Qwen for this analysis
http://localhost:6333/dashboard	Qdrant's own view of the collections and points

Show the tutor (4 minutes)

1. **Vectors & RAG** → *One search, step by step* → *"How are internships counted in years of experience?"* → *Search step by step*: the vector, the cosine list, then the reranked list — point at a chunk that moved up (↑) and the KB-RR-003 section kept first.
2. *2D map* → `hr_knowledge` colored by `document_id`: each document's sections form a group.
3. **Matching internals** → GenAI Engineer + a candidate → *Show what the analysis uses*: pick a requirement and show the snippets with both scores; then the 3 HR searches; then open the prompt.
4. **Search** → *HR knowledge base* → an English question, *"How long do we keep the data of a rejected candidate?"*: it should find the French policy KB-HR-004 (cross-lingual; check it during your rehearsal).

Numbers to quote

99 knowledge chunks from 15 documents (3 in French); 1,024 dimensions; 20 HR chunks (or at least 30 CV/job snippets) retrieved, reranked, and the best 2–5 kept. Retrieval evaluation: 40 questions (10 in French, some cross-lingual), 4 configurations (dense, + reranker, + role filter, both), Hit@1/3/5 and MRR — Kaggle job `rag`.

Questions the tutor may ask

- *Why BGE-M3?* — Multilingual and strong on retrieval; French CVs and policies match English jobs and questions.
- *Bi-encoder vs cross-encoder?* — BGE-M3 encodes the question and the text separately (fast, comparable over millions of points, approximate); the cross-encoder reads both together (precise, slower): so it only rescores the 15–30 points found first.
- *Why chunk by section?* — One section = one rule; with the document title in front, it is self-explanatory. The knowledge collection is recreated from the files at each *Set up*; stable ids let a re-indexed candidate or job replace its old points instead of duplicating them.
- *What is contextualization?* — Giving the LLM exactly the context it needs: the requirements, the company rules retrieved, facts computed by code (years of experience), and the CV.
- *Why compute the years in code?* — LLMs are unreliable at date arithmetic; the experience rules (overlaps counted once, internships at 50 %, at most one year) are applied exactly.

Code: `src/recruitment_ai/rag/knowledge_index.py`, `rag/search_index.py`, `embeddings/`, `reranking/`, `vectorstore/qdrant.py`, `matching/semantic.py`, `matching/analyzer.py` (`retrieve_knowledge`, `build_prompt`).

---

5. The LLM analysis

Say it: *"Qwen2.5-7B judges every job requirement against the CV and must quote the CV for each judgement; code then checks every quote and computes the grade with the company's rubric, so the result is evidence-backed, reproducible and never a decision: a recruiter decides."*

How it works

1. **Three modes**:
   - *fast semantic matching* (no LLM, seconds): ranks every candidate of a job (`matching/semantic.py`);
   - *full AI analysis* (about a minute per candidate, the Matching page): the evidence-backed report (`matching/analyzer.py`);
   - *the screening agent* (the application pipeline, part 3): the same judgement by Qwen, given the CV passages it retrieved for each requirement, then extra checks by code (below).
2. **Full analysis**: retrieval and contextualization (part 4) → Qwen (base model, deterministic, ≤ 1,800 tokens) answers JSON: for every requirement an **evidence level** — *demonstrated* (used in a job or project), *equivalent* (a transferable skill, e.g. Azure for AWS), *listed* (only in a skills list), *not evidenced* — a **CV quote** (≤ 25 words) and the relevant **years**; plus strengths, gaps, interview questions, a summary.
3. **Checks by code**:
   - every quote must be in the CV (the exact text, or ≥ 60 % of its words): otherwise a *demonstrated* or *equivalent* claim is counted as *listed*, with a warning;
   - a requirement the model skipped is marked *not assessed*;
   - the **grade, score and action come from the rubric** (KB-RR-002, `matching/rubric.py`), not from the model.
4. **The rubric as code**: evidence weights demonstrated 1, equivalent 0.75, listed 0.5, not evidenced 0; importance weights high 3, medium 2, low 1; must-have coverage = weighted average.
   - Grade 3 (strong fit): every must-have demonstrated or equivalent, experience met (1 year tolerance), nice-to-have coverage ≥ 50 %;
   - Grade 2 (partial): ≥ 70 % must-have coverage, or every must-have with experience up to 2 years short;
   - Grade 1 (weak): ≥ 25 %; grade 0: less, or a *hard* requirement (legally or contractually mandatory, e.g. a work authorisation) not evidenced;
   - score = 100 × (0.85 × must-have coverage + 0.15 × nice-to-have coverage), kept inside the grade's band (3: 75–100, 2: 50–74, 1: 25–49, 0: 0–24); action: shortlist (3), recruiter review (2), not recommended (0–1) — never an automatic rejection.
5. **The screening agent's extra checks** (`agents/screening.py`, `verify`): a quote must also *support* the requirement (cross-encoder relevance above the dev-calibrated "equivalent" threshold), else the level comes from the passages retrieved for it; a *demonstrated* quote found only in a skills section or the summary becomes *listed* (rubric definition); a requirement the model left out is raised when the fast matching found strong evidence in a position or project (its "met" was precise on the dev pairs); missing years are counted from the relevant positions. Every change is recorded with its reason, shown in the report. These checks answer the errors of the full analysis on the dev pairs (claims "met" for partial or missing requirements, and the reverse).
6. **If the LLM fails**: the fast semantic estimate is shown, with a warning.
7. **Stored** (`analyses`, status *pending review*), with its **trace** (searches, prompt, raw answer, checks, tokens) → notification (the pipeline: its own step *Notify the recruiters*).

Where it appears

Where	What you see
Matching	the ranking of every candidate in seconds; *Analyse* the chosen ones with the full AI analysis
Reports	the report: grade, score, coverage, requirement table with the CV quote and its level, strengths, gaps, interview questions, rules used; PDF and Markdown downloads; *Under the hood*

Where	What you see
Under the hood → Matching internals	step 4: Qwen's time and tokens, its raw JSON, the quotes rejected, fast semantic vs AI level per requirement
Human validation	the analyses waiting for a recruiter
Under the hood → Database → `analyses`	grade, score, status and the full report JSON with its trace

Show the tutor (3 minutes)

1. **Matching** → *GenAI Engineer*: the ranking in seconds; analyse one candidate.
2. **Reports** → the analysis: the requirement table with quotes; download the **PDF**.
3. **Matching internals** → step 4: the raw answer next to the final levels; point at a quote check.

Questions the tutor may ask

- *Why not let the LLM give the grade?* — Consistency and auditability: the model only judges evidence; the grade follows the written rubric, the same for every candidate.
- *How do you detect hallucinated evidence?* — Every quote is searched in the CV text by code; unsupported claims are downgraded and flagged.
- *What does "not evidenced" mean?* — The CV does not mention it: a question for the interview, not a proven gap.
- *Fairness?* — The prompt forbids protected attributes (age, gender, origin, religion, health, family); the assistant refuses such requests; a recruiter validates everything.
- *How is it evaluated?* — 36 CV-job pairs graded with the rubric (silver labels): grade accuracy, within-one accuracy, quadratic kappa, Spearman, must-have status accuracy; the fast matcher's thresholds are calibrated on dev (Kaggle job `matching`). On the 22 test pairs the full analysis is within one grade 95.5 % of the time (fast matching 81.8 %) but less accurate per requirement (55.1 % vs 60.1 %); the screening agent is evaluated on the same pairs (Kaggle job `screening`).

Code: `src/recruitment_ai/matching/analyzer.py`, `matching/rubric.py`, `matching/evidence.py`, `matching/semantic.py`, `reports/markdown.py`, `reports/pdf.py`, `services/matching.py`.

---

6. Knowledge sources (RAG)

Say it: *"Three knowledge sources feed the AI: the company's HR knowledge base — 15 documents, three in French — the candidates' CVs and the job descriptions, all searchable as vectors; and the key rules are also implemented in code, so they are applied exactly."*

The HR knowledge base (`data/knowledge_base/`, Markdown with metadata: id, title, category, language, roles)

Id	Document	How the system uses it
KB-RR-001	Screening process and role of the AI	the AI never decides; service levels (answer within 10 working days, interview contact within 5) in n8n's messages and digest
KB-RR-002	Fit scoring rubric	**implemented in code** (`rubric.py`); given to Qwen in the analysis prompt
KB-RR-003	Experience counting and seniority	**implemented in code** (`evidence.py`: overlaps once, internships 50 %, at most 1 year)
KB-RR-004	Structured interview guidelines	interview questions; interview invitations
KB-HR-001	Career ladder and compensation bands	answers of the assistant (salary questions)

Id	Document	How the system uses it
KB-HR-002	Remote work policy	answers of the assistant
KB-HR-003	Fair evaluation and responsible AI	the fairness rules of every prompt; refusals
KB-HR-004	Candidate data policy (French)	no CV file kept, consent in the portal, deletion of a candidate, retention in the messages
KB-JP-001…005	Job profiles: ML engineer, NLP/LLM engineer, data analyst/scientist (French), backend developer, DevOps/cloud	accepted equivalences retrieved for analyses (filtered on the job's role)
KB-SD-001/002	Technical skills glossary; transversal skills (French)	skill definitions and equivalences

The other two sources: **candidates** (CV snippets) and **jobs** (requirements and descriptions), in Qdrant (part 4) and PostgreSQL (part 7).

Where it appears

Where	What you see
Search → *HR knowledge base*	a question → the best sections with their document id and score (EN/FR, optional job-profile filter)
Under the hood → **Vectors & RAG**	the `hr_knowledge` collection, its points and its 2D map
Reports → *Company rules used*	the documents given to Qwen for an analysis
AI assistant → *Sources*	the documents, candidates and jobs an answer relied on
`data/knowledge_base/`	the 15 Markdown files

Show the tutor (2 minutes)

1. Open `data/knowledge_base/recruitment_rules/fit_scoring_rubric.md`, then `src/recruitment_ai/matching/rubric.py`: the same rules, as text for the LLM and as code for the grade.
2. **Search** → *HR knowledge base* → *"Combien de temps garde-t-on les données d'un candidat ?"* → KB-HR-004.
3. **AI assistant** → *"How are internships counted in years of experience?"* → the answer cites KB-RR-003.

Questions the tutor may ask

- *How do you update the knowledge?* — Edit the Markdown file, then Dashboard → *Set up*: the `hr_knowledge` collection is recreated from the files.
- *Why Markdown?* — Readable, versioned in git, and its sections are natural chunks.
- *Why also code for some rules?* — Rules that produce numbers (grade, years) must be exact and reproducible.

Code: `src/recruitment_ai/data/knowledge.py`, `rag/knowledge_index.py`, `services/demo.py`.

---

7. Data storage (PostgreSQL)

Say it: *"PostgreSQL holds the system's records — applications with the log of every step, candidates, jobs, analyses, recruiter decisions, notifications and the assistant's traces — while Qdrant holds the vectors; documents produced by the AI are kept whole as JSON next to the columns we filter on."*

How it works

Table	One row =	Main columns
`applications`	an application through the pipeline	`status` (running, waiting_review, closed, failed), `stage`, `steps` (JSON: each step's status, times, result, error), `orchestrator` (n8n, api), `decision`, `resume_url` (the n8n Wait node)
`candidates`	a candidate	`full_name`, email, phone, location, `profile` (JSON), `cv_text`, `source` (upload, demo, portal), `extraction` (JSON)
`jobs`	a job	title, company, `role`, `requirements` (JSON: M1… must-haves, N1… nice-to-haves), status, source
`analyses`	a candidate analysed for a job	`method` (agent, llm, semantic), `grade`, `score`, `status`, `final_grade`, `report` (JSON, with the trace), `application_id`
`reviews`	a recruiter decision	`decision`, `reviewer`, `adjusted_grade`, `interview_at`, `notes`
`notifications`	an event or an n8n message	`event`, `title`, `message`, `payload`, `source` (app, n8n), `read`
`agent_runs`	a question to the assistant	`question`, `answer`, `steps` (tool calls and timeline), `sources`, `seconds`

- The tables are created and updated by **Alembic migrations**, run automatically when the API starts.
- Deleting a candidate also deletes their applications, analyses and reviews (cascade) and their Qdrant points (right to erasure).
- Repositories (`database/repositories.py`) hold the queries; unit tests use the same models on SQLite.

Where it appears

Where	What you see
Under the hood → Database	every table with its rows; open a row to read its JSON (profile, report and trace, steps)
Adminer (http://localhost:8080)	the tables, plus SQL queries (System *PostgreSQL*, server `postgres`)
psql	`docker exec -it recruitment_postgres psql -U postgres -d recruitment_db`
Dashboard	the counts
`docs/reports/observe_the_data.md`	ready-made SQL queries

Show the tutor (2 minutes)

1. **Database** → `candidates` → open a row: the profile JSON.
2. `analyses` → a row: the report JSON and its `trace` (raw answer, checks); `reviews`: the decisions; `agent_runs`: the steps and timeline.
3. Adminer → *SQL command*:

```
SELECT c.full_name, j.title, a.method, a.grade, a.score, a.status
FROM analyses a JOIN candidates c ON c.id = a.candidate_id JOIN jobs j ON j.id = a.job_id
ORDER BY a.created_at DESC;
```

Questions the tutor may ask

- *Why JSON columns?* — Profiles and reports are documents produced and read whole; the fields we filter or sort on (grade, score, status) are real columns.
- *Why PostgreSQL and Qdrant?* — Records and transactions vs similarity search on vectors; Qdrant points carry the candidate or job id that links them to PostgreSQL.
- *How is the schema versioned?* — Alembic migrations in `alembic/versions/`.

Code: `src/recruitment_ai/database/models.py`, `database/repositories.py`, `alembic/versions/20261017_1800_a1b2c3d4e5f6_application_schema.py`, `alembic/versions/20261018_1200_b7c1d2e3f4a5_application_pipeline.py`.

---

8. The n8n orchestrator workflows

Say it: *"n8n orchestrates the recruitment process around the AI: one execution per application runs every step of the pipeline, from the CV to the final decision, and waits for the recruiter; other workflows read job ads, turn the other events into the right messages and send a daily digest, respecting the company's service levels; a failure in any workflow raises an alert."*

How it works

Workflow	Starts when	Its steps
Application pipeline	a CV arrives: portal or recruiter upload (`/webhook/cv-intake`)	one node per step (part 3): intake checks → document processing → candidate JSON → store and index → acknowledgement → answer to the candidate → screening agent → report → notify the recruiters → **Wait** (resumed by the decision; a reminder after 10 days) → final decision → notify the candidate (e-mail node, disabled until SMTP) → close. A failure before the answer is explained to the candidate (error branch)
New job	a job ad is posted (`/webhook/new-job`)	the API structures the requirements with Qwen and stores the job → answer
Recruitment events	the other app events (`/webhook/recruitment-events`, when `N8N_EVENTS_ENABLED=true`): analyses and decisions made from the Matching page	a Code node prepares the message for the event: priority review (grade 2–3), interview invitation, polite answer after a rejection (the recruiter's reason kept internal), shortlist reminder (contact within 5 working days), on-hold reminder (answer within 10) → shown in the app (+ an e-mail node, disabled until SMTP credentials are set)
Daily digest	every day at 9:00, or *Run the daily digest now*	the API's digest (waiting decisions, service levels at risk) → notification
Error alert	any of the others fails	the failure → notification

- The app sends its events in the background: if n8n is stopped, the app still works.
- n8n calls the API at `http://host.docker.internal:8000` (the API listens on `0.0.0.0`).
- The workflows are files in the project (`n8n/workflows/*.json`), imported and published by `python scripts/n8n_setup.py`.

Where it appears

Where	What you see
n8n → *Overview* → *Workflows*	the 5 workflows; open one: the canvas (steps, the explanatory note); double-click a step: its settings
n8n → a workflow → **Executions**	every run, green or red; click a run, then a step: the data that went in and out
Notifications & n8n	which workflows are published (green dot), test event, daily digest, latest runs (with `N8N_API_KEY`)
Notifications	the messages written by n8n (robot icon)
Candidate portal	the candidate's side of the *Application pipeline*
Application pipeline (app page)	the same applications, step by step, from the API's side

Show the tutor (3 minutes)

1. n8n → *Application pipeline* canvas: one node per step, the part of the architecture under each node, the Wait node, the error branch.
2. App → **Candidate portal** → a position, a CV, tick the consent box (KB-HR-004), route *Through n8n* → *Apply* → the candidate's answer.
3. n8n → *Application pipeline* → **Executions**: the run is *waiting*; open *Screening agent (LangGraph)*: the API's JSON (jobs analysed, tool calls, timeline).
4. **Human validation** → decision *Invite to an interview* (with a date) → n8n: the same execution resumes, *Final decision* → *Notify the candidate* → *Close the application* turn green → **Notifications**: *Interview invitation prepared for …*.

Questions the tutor may ask

- *Why n8n?* — Visual orchestration that recruiters' IT can change without touching the AI code, with ready connectors (e-mail, Slack, calendars) and a history of every run.
- *What if n8n is down?* — The API runs the same pipeline steps itself (the portal's direct route, the recruiter upload); events are sent in the background and the app never waits for n8n; if n8n does not answer when a recruiter decides, the API finishes the application; the error workflow reports failures of the workflows.
- *How does n8n wait for a human?* — A Wait node "on webhook call": its resume URL is stored with the application; the recruiter's decision calls it, and the same execution continues.
- *Why are e-mails not sent?* — They need SMTP credentials; the prepared messages are shown in the app instead.
- *Is a rejection ever automatic?* — No: a rejection message is only prepared after a recruiter rejected, with a written reason (KB-RR-001).

Code: `n8n/workflows/*.json`, `scripts/n8n_setup.py`, `src/recruitment_ai/services/pipeline.py`, `services/events.py`, `api/routers/applications.py`, `api/routers/webhooks.py`, `frontend/app_pages/portal.py`.

---

9. If something goes wrong during the demo

Problem	What to do
The Kaggle AI server stopped (red dot in the sidebar)	Show what is stored: Application pipeline (earlier applications), Reports, Matching internals (step 4 of earlier analyses), Assistant internals (earlier runs), Evaluation, Database; these pages keep working without the models
An application stopped at a failed step	Application pipeline → the application → read the error → fix (e.g. restart the AI server) → *Continue the workflow from this step*
An analysis takes long	Show a pre-run one in Reports; Qwen takes about a minute per candidate on the free T4 GPUs
n8n shows "not published"	Open the workflow in n8n → **Publish**
A page shows an error	Read it out: the messages say what to do (e.g. restart the Kaggle notebook and update `.env`)

Be honest about the limits: the match labels are silver (made with the written rubric, not by recruiters); the evaluation samples are small; the AI server lives in a Kaggle session (its link changes each time, 30 GPU hours per week); the extraction numbers are on the dev split (the test split is kept for the final evaluation); the vision-language path (`extraction-vlm`) and the clean-text comparison (`extraction-text`) have numbers only once their Kaggle jobs have run, and so does the screening agent's evaluation (`screening`); the live workflow reads scans with OCR then the fine-tuned text model (Qwen2.5-VL stays an evaluated alternative).